

TECHNICAL REPORT / 01

VC Attention Fusedpipe 与 D 调度优化

B200 / SM100 · 中文技术报告 · 2026-10-04

本报告解释 fusedpipe 的源码调度和 D 的 SASS 重排，保留 cbb2da8 的历史实验数据。当前发行版自 7ae8e21 起默认 mid4 + eligible B200 fusedpipe，D 已关闭。新增公开复现工具与环境记录见第 13-14 页。

层次	具体变化	独立测得的收益
已有 VC 算法	FP8、ExpCast、Tensor Core 分母、inline O rescale 等，构成共同底座。	本报告不重新拆分其贡献
fusedpipe	当前 K descale 提前读取；O rescale 从 4×32 列串行改为 2×64 列预先读取。	1.013014×，相对改动前 VC
D SASS patch	首段 score load 增加独立 scoreboard；两个 FMNMX3 提前到完整读取等待之前。	1.002647×，相对已有 fusedpipe

核心判断：两者都利用已有流水线中的独立工作来隐藏等待。fusedpipe 使用更长的寄存器存活期换取读取与计算的重叠；D 只重排现有指令并修改依赖控制。它们都没有增加 warp，也没有重新设计 softmax 算法。

测量范围

真实 MiniMax-H3 捕获输入，step 24、layer 0/24，S=73426，H=7/56，D=128；传入全部 Q/K/V descale，mid_window_blocks=4，dense、无 skipping。主表为每 case 20 轮配对的中位数之比。计时只覆盖一个 attention kernel，排除量化、V packing、编译、模型执行与通信。

阅读路线

第 2 页给出数学和 warp 上下文；第 3-4 页解释 fusedpipe；第 5-6 页逐条解释 D；第 7-8 页列性能及正确性边界；第 9-10 页说明运行时补丁与参数适配；第 11-12 页补充合成成长序列结果；第 13-14 页给出公开复现和环境；第 15 页列完整补丁字。

历史 BF16 / 最新 D 的四 case 几何平均约为 2.137×，属于跨批次的整体 VC 对比，不是 fusedpipe 或 D 单项收益。不同批次的增量比值不相乘成新的“实测总收益”。

01 数学与已有流水线

设 r_{ij} 为 FP8 Q、K 的原始点积， dQ_i 为当前 Q 行所属量化块的反量化因子， dK_b 为当前 K block 的反量化因子。理想恢复的 logit 为 $\sigma \times dQ_i \times dK_b \times r_{ij}$ ，其中 $\sigma=1/\sqrt{D}$ ，D 是每个 head 的 Q/K 维度，本报告取 128。V 的 descale 用于恢复 V 的幅值。

```
c_i      = sigma * dQ_i * log2(e)
tilemax  = dK_b * max_j(r_ij)
alpha_i  = 2 ** (c_i * (M_i - M_i_new))
A_new    = alpha_i * A_old + P_block @ V
l_new    = alpha_i * l_old + sum(P_block)
O_final  = dV * A_final / l_final
```

A 是尚未归一化的输出累加器，I 是分母。实现中这块 TMEM 累加器也叫 O；后文“O rescale”实际是对 A 和 I 同时乘 alpha。M 的更新和既有 deadband 策略不变。上式用于说明 online softmax 的缩放关系，实际 P 仍走已有的 FP8 ExpCast 编码与 Tensor Core 分母路径，并非精确 BF16 softmax。

为什么要同时 rescale 输出与分母

新 block 若提高运行中的行最大值，旧 P 对应的指数基准便发生变化。旧分子与分母必须同时乘 alpha，最终的 A/I 才保持一致。fusedpipe 仅改变这些数从 TMEM 读入寄存器的时间和分片方式，没有改变 alpha 或更新公式。

两项优化发生在哪里

参与者	当前职责	本次是否改变
softmax warps 0-7	score TMEM 读取、row max、ExpCast；其中已有 inline correction / epilogue 分工。	只调整内部 schedule
warps 8-9 / 10 / 11	空闲或服务角色 / 数据 load / MMA；全 CTA 共 12 warps、384 threads。	warp 数和分工不变
MMA 与 softmax 间同步	S_full 发布 QK score；P_full_O_rescaled 发布首段 P 和已 rescale 的 O。	同步协议不变

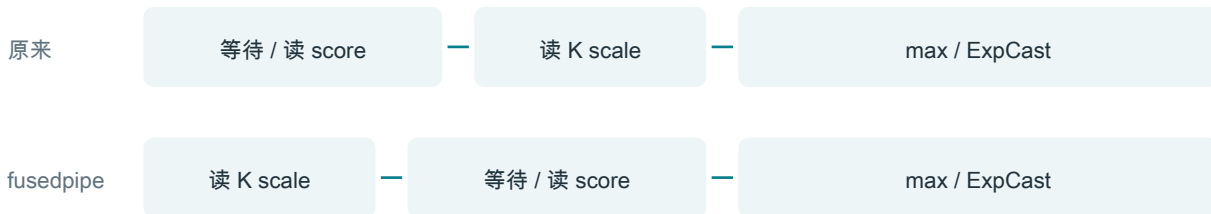
QK / PV MMA、ExpCast 的 FFMA/F2FP/PRMT 编码、QKV 量化、V packing 与扫描次序均不由 fusedpipe 或 D 改写。已有 ExpCast 对 SFU 的替代不能算作这两项新增收益。

[S1] [warp 与寄存器角色设置 · L807](#)
[S2] [ExpCast scale 与 bias 的构造 · L3907](#)

02 Fusedpipe : 提前读 K descale

原路径先等 QK score 就绪、取回 score、完成读取并处理 mask，随后读取当前 K block 的 descale。

fusedpipe 把同一次标量读取放到 S_full 等待之前。地址中的 batch、kv_head 和 n_block 此时已经确定，读取的 descale 在 kernel 执行期间保持不变。



逻辑发射顺序示意；框宽不代表真实周期，异步读取可以与后续等待重叠。

```

# Before
wait(S_full[stage])
scores = load_score_TMEM()
wait_score_loads_and_arrive_score_empty()
apply_mask(scores)
k_scale = K_descale[batch, kv_head, n_block]

# Fusedpipe
k_scale = K_descale[batch, kv_head, n_block]
wait(S_full[stage])
scores = load_score_TMEM()
wait_score_loads_and_arrive_score_empty()
apply_mask(scores)
  
```

它隐藏的是什么等待

这次 load 与 QK MMA 完成没有数据依赖，可以先进入访存流水线。softmax 随后等待 S_full 时，标量读取有机会同时推进。当前 canonical SASS 里可见 0xa6f0 的 LDG.E 位于 0xa710 的 SYNCS.PHASECHK...TRYWAIT 之前。它读的是当前 n_block，不能描述成“提前读取下一 block”。

为什么不改变数值或同步

消费点仍是同一个 tile 的 row max 与 ExpCast；读取的索引、数值和次数不变。所有 score TMEM 读取依然在 S_full 之后，score_empty 的发布也保留。K descale 不来自正在被 MMA 写入的 score 缓冲，因此提前加载不绕过生产者-消费者关系。

这是逻辑上同一次标量 load 的前移，不代表增加预处理或减少全局读字节。缓存、合并和广播会影响物理访存量。源代码证明了可重叠性；现有 A/B 没有单独证明全部收益只来自这一条 LDG。

[S3] fusedpipe gate、K descale 提前读取和 S_full 等待 · L3665

[S4] 非 fusedpipe 的原 descale 读取位置 · L3792

03 Fusedpipe : O 改成 2×64 列

旧 correction helper 将 128 列 O 分成四个 32 列片段，每片读取后立即乘 alpha 并写回。新 helper 先发起两个独立的 64 列读取，再处理第一半；这样，第二半的读取可以在第一半的乘法与写回期间推进。

```
# Before: four dependent chains
for cols in [0:32, 32:64, 64:96, 96:128]:
    x = load_TMEM(O[cols])
    store_TMEM(O[cols], x * alpha)

# Fusedpipe: issue both reads before dependent multiply
a = load_TMEM(O[0:64])
b = load_TMEM(O[64:128])
store_TMEM(O[0:64], a * alpha)
store_TMEM(O[64:128], b * alpha)

# Unchanged: denominator is still read LAST
l = load_TMEM(denominator)
store_TMEM(denominator, l * alpha)
fence_TMEM_stores()
```

canonical SASS PC	操作	意义
b210 / b230	LDTM.x64 / LDTM.x64	两个 O 半片先发起读取
b250	首个 FMUL2	第一次消费 O 数据
b470 / b660	STTM.x64 / STTM.x64	依次写回两半 O
b670 / b680 / b690	分母 LDTM / FMUL / STTM	分母仍在两半 O 之后
b6a0 / b6e0	FENCE / SYNC.S.ARRIVE	写入排序后才发布 P_full_O_rescaled

每个发生 rescale 的逻辑行仍读取并写回 128 个 FP32 O 元素及 1 个 FP32 分母，即 516 B 读取 + 516 B 写入。字节数不变；x64 减少分片并扩大可并行的读取窗口，代价是更多数据同时保留在寄存器中。

首次 tile 不读未初始化 O；只有原有 warp ballot 判断需要 rescale 时才进入 helper。首段 P 的发布仍在 O 与分母完成写回和 fence 之后。伪代码表达依赖关系，不保证与编译器排出的每一条 FFMA2 顺序完全一致。

[S5] 两半 O 预先读取及分母最后处理 · L4734

[S6] 首段 P、条件 rescale 与发布同步 · L4027

04 D：提前消费首段 score

D 直接修改 fusedpipe 编译产物中两个重复 softmax body，各改 5 个现有的 16 B 指令槽，共 10 槽、74 个字节。四条 score LDTM.x32 的发射 PC 全部保留。改变的是首条 load 的依赖标记，以及两个 max 叶节点与 R2UR/NOP 的位置。

body 1 PC	原始 fusedpipe	D
a750	LDTM.x32 R36, tmem[UR10]	同一 load，增加 write B1
a760-a780	其余 3 条 score load；末条 write B0	完整指令字不变
a790	R2UR UR9, R0	FMNMX3 R77,R36,R37,R38,!PT；wait B1
a7a0	NOP	FMNMX3 R76,R39,R40,R41,!PT
a7b0	ARRIVE score_empty；wait B0	PC 与完整指令字不变
a7c0	第一个 FMNMX3，输出 R77	R2UR UR9, R0
a7d0	第二个 FMNMX3，输出 R76	NOP

读取粒度与等待粒度分开

首条 x32 load 给出 R36-R67；后面三条分别给出 R4-R35、R164-R195、R132-R163。两个被提前的 FMNMX3 只使用首段中的 R36-R41，因此无需先等所有 score 段完成。D 给首段单独挂一个 dependency slot，首个 MAX 等该段就绪后即可执行。

```
Original: LD0 LD1 LD2 LD3 | R2UR NOP | WAIT_ALL/ARRIVE | MAX0 MAX1
D:        LD0 LD1 LD2 LD3 | WAIT_0/MAX0 MAX1 | WAIT_ALL/ARRIVE | R2UR NOP
```

B0/B1/B2 是硬件 scoreboard 的依赖槽，不是 shared-memory mbarrier 编号。“write B1”表示记录异步结果的完成依赖，不表示向内存写入。原来的完整 score 等待仍约束 ARRIVE，producer 不能因此提早覆盖尚未读完的 score。

精确收益来源是：首段 score 就绪后，两个现有 MAX 可以先做；与此同时，其余 score 段仍可能在返回。D 没有在四条 load 之间插入 MAX，没有省掉任何 score 读取，也没有删掉完整读取等待。

[S7] 完整 10 槽补丁表 · L25

05 D：第二 body 与依赖安全性

body 2 PC	原始 fusedpipe	D
dea0	LDTM.x32 R36, tmem[UR6]	同一 load，增加 write B2
deb0-ded0	其余 3 条 score load；末条 write B1	完整指令字不变
dee0	R2UR UR5, R0	FMNMX3 R76,R36,R37,R38,!PT；wait B2
def0	NOP	FMNMX3 R75,R39,R40,R41,!PT
df00	ARRIVE score_empty；wait B1	PC 与完整指令字不变
df10 / df20	两个原 MAX 的位置	R2UR UR5,R0 / NOP

为什么第二段不能照抄 B1 或 B0

两个 body 的活跃依赖不同。第二 body 的 B1 已用于完整 score 读取；B0 还保护 0xde40 的 descale 异步读取，并在 0xdf70 的相关覆盖前等待。D 因此选已退役的 B2；其可用性需要沿 0xde60 的 waitmask=6 及慢速等待分支一起核对，不能只看紧邻 load 的几条指令。

复核项目	结论与依据
MAX 数据与数值	输入寄存器、!PT 修饰和归约树不变；后续 a880 / dfd0 的合并节点不变。
移动后的寄存器冲突	检查每个 body 的 8 对逆序关系，无 RAW/WAR/WAW 冲突。结果寄存器不覆盖 raw score。
R2UR 延后是否合法	UR9 / UR5 的首次后续消费者仍在 b1b0 / e8f0，初始化未跨越消费者。
依赖槽是否已释放	包含入口与慢速等待返回路径；第一 body 验证 B1，第二 body 验证 B2，同时保留已有 B0。
同步与元数据	全部 33 个 ARRIVE 的 PC 和指令字不变；对应 0x3904 元数据记录不变。

ARRIVE 的位置和编码保持不变，不等于它在运行时执行的周期不变；让等待附近的有用工作更早完成，正是调
度优化的目的。二进制长度、分支目标、寄存器分配元数据和内存访问集合都没有增加。

现有依据由完整机器码 diff、控制流 / 活跃寄存器分析，以及相同 FP8 输入下的逐字节输出比较构成。上述检查支持当前验证过的
binary family；不能自动推广到另一版编译器生成的代码。

06 性能：分别量化两项增量

GPU 为 B200；所有行 S=73426、D=128。I00 / I24 表示 layer 0 / 24；H7 为真实 H56 捕获张量的前 7 个 head，不是一次实际 CP8 推理。表中时间为 20 轮中位数，倍率=对照中位数/候选中位数。四 case 等权取倍率的几何平均。

fusedpipe 对改动前的实验 v4：1.013014×

case	原 VC ms	fusedpipe ms	倍率	配对获胜
I00-h7	6.451459	6.401475	1.007808x	17/20
I00-h56	56.680652	55.950128	1.013057x	20/20
I24-h7	6.392039	6.287898	1.016562x	20/20
I24-h56	56.353769	55.540204	1.014648x	20/20

独立的 12 轮批次得到 1.014248×、48/48 配对获胜；主表 20 轮批次为 77/80。可表述为这些输入上约 1.30%-1.42% 的速度倍率增益，不能将两个批次混合或逐 shape 挑选较好批次。

D 对已经包含 fusedpipe 的同一产物：1.002647×

case	fusedpipe ms	加 D 后 ms	倍率	配对获胜
I00-h7	6.365821	6.344682	1.003332x	13/20
I00-h56	55.842206	55.718091	1.002228x	17/20
I24-h7	6.266525	6.244125	1.003587x	14/20
I24-h56	55.224094	55.144452	1.001444x	12/20

独立首轮 12 次测试为 1.002887×、37/48 获胜；20 轮确认测试为 56/80。D 的重复汇总收益约 0.26%-0.29%，属于小幅增益，不能声称所有时间段或任意 shape 均稳定获益。

计时与归因边界

CUDA events 计时五次原生 attention kernel 的 replay；量化、V packing、分配、编译和校验均在计时外。所有纳入的记录已完成且无 foreign PID / guard failure；时钟未锁定。这里没有独立的 NCU 因果分解，调度重叠是由源码和 SASS 支持的机制解释。

fusedpipe 的两项源码变化作为一个整体测量；本表不能分别给 K descale 前移与 O2×64 分配百分比。D 的对照已经有 fusedpipe；两组来自不同批次，不能把倍率相乘并称为新实测。

07 BF16 对照、资源与正确性

整体 VC 对 BF16：历史时间计算，约 2.137×

BF16 使用 bf16_ref，源 revision 为 2cae9072，使用原始 BF16 QKV 和默认扫描顺序；VC 使用预处理后的 FP8、全部 Q/K/V descale 与 mid4。BF16 时间来自 10 月 2 日的 20 轮批次，D 时间来自 10 月 3 日独立确认批次。

case	BF16 ms	VC + D ms	历史 BF16/VC
I00-h7	13.656083	6.344682	2.152367x
I00-h56	114.823511	55.718091	2.060794x
I24-h7	13.896109	6.244125	2.225469x
I24-h56	116.539191	55.144452	2.113344x

这不是新一轮同批次 BF16/D 配对，也不是 D 单独带来 2.14×。它包括整个 VC 算法及此前优化，且只代表 kernel 时间；不能据此推出整个 inference pipeline 的加速比。

寄存器：D 保持分配，fusedpipe 扩大在途数据

指标	fusedpipe → D	应如何理解
资源元数据 REG / STACK	168 / 8 → 168 / 8	工具报告字段；不是 softmax warp 实际活跃数。
softmax USETMAXREG	232 → 232	该角色的寄存器配额上限，不等于实际使用量。
CFG 静态 GPR 活跃峰值	205 → 205	静态分析；不能当成硬件测得的寄存器用量。
静态 LDL / STL 指令数	3 / 2 → 3 / 2	已有 epilogue spill 保留；未增加热循环 spill。
代码大小 / 指令槽	87040 B / 5440 → 相同	D 重排现有槽位；未插入新代码。

同一组冻结产物中，改动前 VC 到 fusedpipe 的保守静态 CFG GPR 峰值为 169 → 205；初始分配和 softmax 额度不变。这反映了两半 O 同时驻留的代价，不能解释为运行时增加 36 个寄存器。D 仅延长两个 max 结果的存活时间，峰值仍为 205。SHARED=1024 不含动态共享内存。

正确性结论的范围

调度候选与各自同输入、同配置的 FP8 对照做逐字节输出比较。最新窗口发布验证 10/10 case，通过 120 次全输出比较和 1670 次 ownership 检查。相同结果说明这些调度改动未引入观察到的数值变化；不等于 FP8 与 BF16 相等，也不是完整模型质量评价。

08 补丁如何进入实际执行路径

D 是每个已编译 callable 的内存补丁，位于 cute.compile 之后。适配器导出 host object，找到其中的 CUDA ELF，在严格校验后替换指令，使用 BinaryExecutionEngine 与 TVM FFI 重新绑定相同 ABI 的函数。它保留原对象及相关生命周期，不修改原始磁盘编译缓存，也不设置全局驱动 hook。

```
cute.compile(...)
-> export host object
-> locate the single embedded CUDA ELF
-> verify architecture + code family + metadata
-> rewrite 10 instruction slots in memory
-> load / bind the same function ABI
-> cache the wrapped callable

unsupported layout -> return original callable
```

当前验证规则

检查层	实际约束
架构与工具链	SM100 ELF flags、ELF64 little-endian；适配器要求 CuTe DSL 4.6.0 与受支持 FFI 包装类型。
机器码	单个目标 text，87040 B；归一化前/后全 text SHA 均验证。
窗口立即数	仅 4 处已验证 scan 常量字段可变化，并且必须等于调用方的 mid_window_blocks。
同步和执行元数据	完整指纹约束资源、常量、符号、重定位及相关执行元数据；仅允许内核命名等已定义差异。
修改范围	10 个原指令字逐个完全匹配后才写入；完成后验证目标 SHA 和二进制长度。

patch revision 为 sm100-score-two-body-window-v2。运行时记录实际 text SHA、归一化 SHA、window 和 applied 状态。版本号本身不能证明 D 已执行；外部 AOT loader 或绕过此适配器的导出路径，需要另外检查实际加载产物。

不支持的 layout 会保留原 callable；真正执行 kernel 时发生的错误不会被当成“不支持补丁”悄悄吞掉。export_to_c 路径也重新进行带窗口绑定的补丁校验。

[S8] 窗口绑定、归一化与全机器码验证 · L142

[S9] 每个 compiled callable 的加载适配器 · L1

09 哪些参数会影响补丁适配

不能用 seqlen 或某个 window 的白名单代替机器码判断。参数有时只是运行时数据，有时会改变 Python/CuTe 的编译分支、寄存器分配、控制流或调用 ABI。当前策略只放行已经证明安全的常量字段变化，其余变化均要求整份产物匹配。

因素	为什么可能改变 kernel layout
mid_window_blocks	编译期扫描常量；常见值仅改变 4 个立即数字段，但 0/1 会产生不同布局，当前 D 回退。
S_q / S_k、batch、heads	可能触发 q_stage、packing、split-KV、布局/stride specialization 等分支；同一分支内也可能仅改变运行时量。
D / Dv、tile、stage、实际 warp 配置	直接改变展开长度、寄存器集合、TMEM/SMEM 布局及同步参与者。
dtype、descale 是否存在和 rank	改变算术与读取路径；有无 descale 不能共用未经验证的补丁。
mask / score mod、skip、LSE、计数器	可能增加控制流、输出或副作用；V-Smooth / SVD / NVFP4 等也属于另一种 specialization。
架构、编译器及选项	SM100/SM103、DSL、ptxas、优化选项均可能改变指令编码和调度控制。

目前实际支持到了哪里

CPU 产物验证覆盖 window=2/3/4/7/8/16/31/1024；0/1/None 的 D 保留回退。fusedpipe 的源 gate 已允许非 None 的非负窗口，所以 mid0/1 可以用 fusedpipe、同时不使用 D。mid8 的 shape 验证为 [32769,7,128]、[73426,7,128]、[73426,56,128]、[188214,7,128]；不是这些 S 与 H 的全部组合，更不是所有 shape 的保证。最新窗口 GPU 对比中，mid2/8/16/1024 同时启用了 fusedpipe gate 放宽和 D family 支持，测得的是组合变化，不能把其全部收益归给 D；None 和 mid4 两侧 native text 完全相同，其微小时间差是回归检查中的波动。

更通用的下一步：控制流与角色匹配

已分析的 mid0/1/4 拥有相同连接关系的 370 个 basic blocks、550 条边；局部 PC 偏移并不等于算法改变。未来可匹配 load / max / arrive 的数据角色与 CFG，再证明 dependency slot 已退役、延后 UR 写不跨消费者、区域内部无额外入口以及元数据有效。该通用 matcher 尚未实现，不能用短指令串匹配直接替代目前的全产物校验。

TECHNICAL REPORT / 11

10 固定种子长序列扩展

这批输入由 CPU torch.randn 直接生成 BF16 Q/K/V，三个独立 Generator 的种子依次为 20261004、20261005、20261006。S=188214/262144，H=7/56，D=128；传入全部 descale，mid4、非因果、无 skipping。它们与前述真实捕获输入属于独立批次。

四路同批次中位数，单位 ms

[S,H,D]	pre-fusedpipe	fusedpipe	fusedpipe+D	BF16
[188214, 7, 128]	49.304358	48.244330	48.137207	94.809500
[188214, 56, 128]	402.057336	394.094482	393.837146	763.531982
[262144, 7, 128]	96.323181	94.386279	94.313400	185.550647
[262144, 56, 128]	781.894336	771.211157	772.152124	1480.788721

分别计算的收益与配对胜率

[S,H,D]	fusedpipe 增量	D 增量	BF16 / (VC+D)
[188214, 7, 128]	1.021972x / 12/12	1.002225x / 9/12	1.969568x / 12/12
[188214, 56, 128]	1.020205x / 12/12	1.000653x / 8/12	1.938700x / 12/12
[262144, 7, 128]	1.020521x / 12/12	1.000773x / 7/12	1.967384x / 12/12
[262144, 56, 128]	1.013852x / 12/12	0.998781x / 8/12	1.917742x / 12/12

比值依次为 pre-fusedpipe/fusedpipe、fusedpipe/D、BF16/D；每项末尾为候选较快轮数。四个 shape 等权几何平均：fusedpipe 为 1.019133x，D 为 1.000607x，整体 VC+D 相对 BF16 为 1.948230x。均为同批次的 kernel-only 数据，不包含量化、V packing、模型执行或通信。

D 在 [262144,56,128] 上的中位数延迟增加约 0.122%，同时配对胜数为 8/12。中位数之比与胜率刻画不同方面，不能据此宣称 D 稳定改善或稳定退化。这里只有一组固定 seed，尚无独立重复批次。

10 长序列方法与证据

项目	实际边界
计时	每 case 12 个完整配对轮次，每样本 5 次 CUDA Graph replay；2 次预调用、每实现 1 秒预热。四路各占每个顺序位置 3 次。
排除工作	输入生成、量化、V packing、分配、编译、正确性验证、profiling、模型与通信。
数值与归属	四个 case 均通过完整输出字节比较及单 kernel 检查；污染和中断会使整次 attempt 无效。
频率与重复	没有锁频；每个 shape 一组接受批次，没有独立重复批次。
公开补充	同目录 JSON 提供 192 个原始 timing samples、seed、输入 hash、配置、顺序统计与产物指纹。

实际测量的是哪些实现

四路分别是匹配的 pre-fusedpipe 控制、fusedpipe、由同一个 fusedpipe 产物派生的 D，以及同批次新测 BF16。历史控制为冻结实验 kernel，共享 H7 packing gate，并非直接运行未经修改的公开 checkout。

四个长 shape 复用三个动态 native 函数：BF16、pre-fusedpipe、fusedpipe；D 是最后一个的二进制派生。fusedpipe 的 text 和 D guard 对应 canonical 87040 字节 family。S=262144 的通过记录来自 typed-FP8 AOT replay，不证明任意 public API / real-Uint8 descriptor 配置均可执行 D。

CPU 产物校验、隔离的 host/internal symbols 和单 kernel callable 标识了 D 派生；这批测试没有新的 device-code dump。FP8 调度候选与匹配控制逐字节相等；不要求与 BF16 相等，也不构成模型画质结论。

原始公开文件：VC-Attention-Long-Sequence-Results.json。拒绝的 attempt 不参与任何统计；拒绝数量保留。历史表格不因复现工具的新增而重写。

11 公开复现入口

tools/report_reproduction 会从当前 checkout 创建私有 source tree，在 CPU 上生成 AOT 对照产物；公开推理 API 的默认值保持 mid4 + fusedpipe，D 关闭。

路线	构建方法
pre_fusedpipe	仅把 staged v4 的 use_fusedpipe 设为 False；保留 mid4、全部 descale 和共同的 B200 packing gate。
fusedpipe	同一份 v4 源码复制到独立 namespace，启用 fusedpipe。
bf16	使用固定 baseline 快照和原始 BF16 输入。
d，显式 opt-in	从同一 fusedpipe object 派生，校验完整 text/metadata，隔离 host symbols；不匹配时失败。

报告里的 pre_fusedpipe 固定 mid4；公开 CLI 的 vc_v4 则保留 original scan（None）。比较时应使用这里的 matched controls，不能把这些名称当成同义词。

```
python tools/report_reproduction/build.py --output build/report
python tools/report_reproduction/benchmark.py \
  --build build/report --shape 32769x7x128 \
  --output results/report-smoke.json
python tools/report_reproduction/summarize.py \
  results/report-smoke.json
```

长序列把 shape 改为 188214/262144 × 7/56 × 128，分别运行。默认 12 轮、5 次 replay、2 次预调用、1 秒预热。需要历史第四路时，在 build 与 benchmark 两步均显式添加 --include-d；校验失败不会静默回退后仍标为 D。

runner 在 capture 前量化和 pack V，只捕获 native attention call。计时前后做完整字节比较，并在计时外 profile 验证恰有一个 attention kernel。普通 vc-attn-bench 包含 V packing，属于另外的 operator 测量范围。完整命令见 docs/reports/reproduction.md。

12 环境与验证边界

组件	历史接受记录中的值
GPU / driver	B200 / SM100 ; driver 590.48.01
Python / Torch	Python 3.12.3 ; Torch 2.11.0+cu130 ; CUDA runtime 13.0
DSL / Triton / quack	4.6.0 / 3.6.0 / 0.6.1
CUDA Python / linker	cuda-python 13.0.3 ; GCC 13.3.0 ; objcopy 2.42
频率	未锁频 ; 每 case 起止 clocks、pstate、power 快照已补入公开 JSON。

historical-environment.json 来自原始接受记录，保留来源 hash。历史 ptxas binary 身份及完整 compiler environment 没有保存，仍明确标为未知，不用今天机器上的值倒填。

新构建自动记录

manifest.json 保存 package 版本、可发现的编译/链接工具及 bundled ptxas 候选、编译选项、runtime library hash、源码变换、ABI/cache metadata、产物 hash 和 D patch proof。发现某个 ptxas 候选不等于证明 DSL 调用了它。构建与运行需使用同一依赖环境。

正确性与 GPU 使用

FP8 调度路线必须输出 finite、与匹配控制逐字节相等；direct/graph 及计时后输出再次比较。BF16 独立检查 finite 与 direct/graph 一致性。runner 在准备阶段和每个计时样本前后检查外部 CUDA PID；GPU 分配由调用者负责。允许共享时结果明确标记 non-isolated。

当前发布与实验范围

D 仍有未解决问题，只有复现工具显式开启它；本报告不证明所有 ABI/shape 的 D 路径、模型画质、训练或端到端加速。新工具的验证单独记录在 reproduction-validation.json，历史计时与新 smoke 不合并。真实捕获张量不公开；合成长序列可重新生成并核对输入 hash。

附录 完整 10 槽补丁与指纹

下表为 canonical mid4 的 text-relative PC。每行用 128-bit 十六进制整数表达完整指令字，ELF 按 little-endian 存放。表中未列出的槽位不变；window family 只在识别阶段归一化 4 处立即数字段，不覆盖调用方实际 window。

a750	OLD 000fe400082c0000000000a002479ee NEW 000e6400082c0000000000a002479ee
a790	OLD 000fe200000e00000000000000972ca NEW 002fe4000780002600000025244d7276
a7a0	OLD 000fe2000000000000000000007918 NEW 000fc4000780002900000028274c7276
a7c0	OLD 000fe4000780002600000025244d7276 NEW 000fe200000e00000000000000972ca
a7d0	OLD 000fc4000780002900000028274c7276 NEW 000fe2000000000000000000007918
dea0	OLD 000fe400082c00000000006002479ee NEW 000ea400082c00000000006002479ee
dee0	OLD 000fe200000e00000000000000572ca NEW 004fe4000780002600000025244c7276
def0	OLD 000fe2000000000000000000007918 NEW 000fc4000780002900000028274b7276
df10	OLD 000fe4000780002600000025244c7276 NEW 000fe200000e00000000000000572ca
df20	OLD 000fc4000780002900000028274b7276 NEW 000fe2000000000000000000007918

归一化 text 与元数据 SHA-256

before	3f8c9a6f08115780f331d674a3effa774 10d88bd2fe9ee91c9fffeed74367591
after	040efd30afd2544024a7a36046ad8382d 4687d2d3d6ce6da9fee4d60b0b6755f
meta	9ab3d51a5555fc0a52621a3623fe36dd d0fa20d1a1802bfd5765ac6e23946e42

4 处 scan 立即数 PC：1740、5fc0、7ca0、a570；仅各指令 bits 32-63 参与 window 归一化。本文所有 PC 均对应所列 canonical 产物，不是其他编译版本的通用地址。